AI MICRO-TRADING AGENT

Antigravity Phase-Wise Prompt Pack

Complete development workflow from project bootstrap to live-readiness

Primary development roles

Antigravity IDE / CLI: Project Commander, architect, reviewer, integration lead, UI/UX lead, QA coordinator.

Codex CLI: Core implementation worker for the deterministic trading engine, backend, tests, adapters, backtester and AI integration.

User: Final authority for architecture changes, broker connection, capital limits and live-trading approval.

Locked principles

Local-first • ■0 additional development spend • deterministic safety layer • independent Position Watcher • paper trading before live • broker abstraction • configurable costs • no LLM in emergency-exit path • UI clearly separates system facts from AI analysis.

How to use this PDF

Run the phases sequentially. Give the phase prompt to Antigravity. Antigravity should inspect the repository, execute or delegate only the requested scope, verify the result, and update project control documents. Do not skip verification gates.

> **Important:** Antigravity should not blindly code the whole project in one request. Use one phase at a time and require a checkpoint at the end of every phase.

Recommended control loop

1. Antigravity plans → 2. Codex implements the assigned coding task → 3. tests run → 4. Antigravity reviews → 5. Codex fixes only approved findings → 6. tests run again → 7. Antigravity updates documentation → 8. user approves phase completion.

Antigravity execution mode

For complex work, use plan/read-only investigation first. Antigravity's current CLI supports plan mode, artifact review, terminal execution, and subagents. Keep file-edit approvals enabled during architecture and live-trading phases.

One-writer rule

> Never let Antigravity and Codex edit the same files concurrently. If Antigravity delegates a coding task to Codex, wait for Codex to finish, then have Antigravity review the diff before another writer starts.

Can Antigravity handle Codex?

Yes, **as an orchestration layer, provided Codex CLI is installed and available in the project environment**. Antigravity can operate through the local terminal and can run multi-step agent workflows. However, the current Antigravity documentation does not establish a special native 'Codex integration' that should be assumed. Treat Codex as an external CLI worker that Antigravity may invoke through the terminal.

Best arrangement

Role	Owner	Responsibility
Commander	Antigravity	Plan phase, inspect repository, maintain specs, delegate, review, verify.
Coder	Codex CLI	Implement assigned code changes and tests.
Reviewer	Antigravity	Review diffs, architecture, tests, performance and failure handling.
Final authority	User	Approve architecture changes and any move toward live trading.

Do not give Antigravity permission to independently enable live orders. Live trading remains a human approval gate.

PHASE 00 — Project Bootstrap & Control Plane

```
You are the Project Commander for the AI Micro-Trading Agent.

Read all supplied project PDFs and repository files before changing anything. Do not implement trading logic yet.

Create a clean project control plane:
- ARCHITECTURE.md
- PROJECT_SPEC.md
- DEVELOPMENT_PLAN.md
- TASKS.md
- DECISIONS.md
- CODING_RULES.md
- TEST_PLAN.md
- DATABASE_SCHEMA.md
- API_CONTRACTS.md
- TRADING_RULES.md
- FAILURE_HANDLING.md
- BROKER_ADAPTER_SPEC.md
- UI_UX_SPEC.md
- AI_ROLES.md

Define the locked architecture, ownership boundaries, one-writer rule, phase gates, and approval gates.

Create the initial repository structure for Python/FastAPI/SQLite/React without implementing business logic.

Inspect the environment and report installed versions for Python, Node, Git, Ollama, Qwen3 availability, DeepSeek-R1 availability, and Codex CLI availability.

If Codex CLI is available, prove that Antigravity can invoke it with a harmless read-only command. Do not let Codex edit files yet.

STOP and produce a bootstrap report. Do not begin Phase 01 automatically.
```

PHASE 01 — Architecture Lock & Technical Design

```
Act as Lead Architect.

Re-read all project control documents and validate the architecture end-to-end.

Lock these boundaries:
Market Data → Normalizer → Feature Engine → Momentum/Early-Move Strategies → Candidate Ranker → AI Queue
→ Deterministic Trade Gate → Execution → Position Watcher → Journal/Performance.

Qwen3 proposes market thesis/evidence.
DeepSeek-R1 challenges the thesis.
Deterministic code makes the final risk/execution decision.
LLMs never own emergency exits.

Define interfaces, data contracts, event schemas, timing measurements, failure modes, and module
responsibilities.

Do not implement trading logic yet.

If any change is necessary, record it in DECISIONS.md and mark it as requiring user approval if it changes
the locked architecture.

Deliver the final architecture package and a dependency-ordered implementation plan.
```

PHASE 02 — UI/UX Design System

```
You are now the UI/UX Lead.

Do not build the full dashboard yet. Design the complete trading-terminal experience.

Create UI_UX_SPEC.md covering:
- design language
- dark-first professional trading UI
- typography
- spacing
- navigation
- responsive behavior
- cards
- tables
- charts
- status badges
- alerts
- drawers/modals
- loading/error/empty states
- accessibility
- real-time update behavior

Define these screens:
1 Command Center
2 Market Scanner
3 Candidate Feed
4 Trading Chart
5 Position Monitor
6 Orders & Execution
7 AI Analysis
8 Trade Journal
9 Performance Analytics
10 Backtesting
11 Risk & Capital
12 System Health
13 Logs/Events
14 Strategy Management
15 Settings

Explicitly separate DETERMINISTIC SYSTEM FACTS from AI ANALYSIS.

Define visual states for BUY, SELL, HOLD, WATCH, RISK, ERROR, CONNECTED, DISCONNECTED, AI THINKING, AI
UNAVAILABLE, PENDING, PARTIAL FILL, FILLED, CANCELLED and REJECTED.

Create wireframes or UI artifacts where useful.

Do not connect the UI to live trading.

Finish with a UI implementation task list for Codex.
```

PHASE 03 — Market Data & Normalization

```
Delegate the implementation of Phase 03 to Codex CLI.

Codex task:
Implement only the market-data foundation defined in the approved architecture.

Requirements:
- provider/broker-neutral MarketData interface
- configurable provider adapter
- normalized market event schema
- timestamps and source metadata
- stale-data detection
- reconnect handling
- rate-limit/error handling
- deterministic test fixtures
- no live order placement
- no LLM calls

Add unit/integration tests and clear logs.

After Codex completes, review every diff. Verify data contracts, timestamps, failure handling and test
coverage. Reject unrelated changes.

Do not proceed to BUY logic until the data layer passes its phase gate.
```

PHASE 04 — Feature Engine

```
Delegate to Codex CLI.

Implement the deterministic feature engine only.

Required features include:
- momentum
- relative volume
- volume acceleration
- price acceleration
- VWAP relationship
- volatility
- spread
- liquidity
- market index direction
- sector strength
- price structure
- regime indicators

Make calculations deterministic, testable and timestamp-safe.

Prevent look-ahead leakage.

Expose a structured FeatureSnapshot consumed by both strategy modules.

Add unit tests, edge-case tests and performance benchmarks.

After implementation, review formulas and data dependencies. Do not add AI reasoning yet.
```

PHASE 05 — BUY Engine: Momentum + Early-Move Detection

```
Delegate to Codex CLI.

Implement two independent strategy modules:

A) Momentum Continuation
- detects an existing move
- validates continuation conditions
- rejects deteriorating momentum

B) Early-Move Detection
- detects developing movement before obvious expansion
- uses price/volume acceleration
- structure
- VWAP
- market/sector alignment
- liquidity

Implement Candidate creation and deterministic ranking.

Use configurable thresholds, not magic numbers.

Create explainable evidence for every candidate.

Do not place orders.

After Codex finishes, review false-positive handling, duplicate candidates, cooldowns, scoring, and tests.
```

PHASE 06 — SELL Engine + Independent Position Watcher

```
This is a critical safety phase.

Delegate to Codex CLI.

Implement:
- hard stop protection
- stale-data emergency behavior
- catastrophic liquidity-loss protection
- daily/capital circuit breaker
- momentum deterioration
- structure break
- spread widening
- market/sector reversal
- time protection
- profit/trailing logic
- AI thesis invalidation as a NON-emergency layer

Create an independent Position Watcher that can operate without Qwen3 or DeepSeek-R1.

The watcher must not wait for an LLM before executing hard protection.

Add tests for model outage, network outage, stale data, partial fills, rejected orders and restart recovery.

Review carefully before proceeding.
```

PHASE 07 — Capital, Risk & Cost Engine

```
Delegate to Codex CLI.

Implement:
- total capital
- available capital
- deployed capital
- realized P/L
- unrealized P/L
- buying power
- position concentration
- configurable autonomy threshold
- minimum economical trade size
- risk limits
- daily loss/circuit breakers
- transaction-cost model

Do NOT hard-code a single brokerage fee.

Costs must be configurable for brokerage, STT, exchange charges, SEBI charges, stamp duty, GST, slippage and other applicable costs.

The initial policy is:
- starting capital: ■100
- autonomous threshold: ■300
- above threshold: human approval

Make both configurable.

Every trade decision must be evaluated on expected NET edge after costs.
```

PHASE 08 — Execution Engine + Broker Abstraction

```
Delegate to Codex CLI.

Implement:
- BrokerAdapter interface
- PaperAdapter
- order lifecycle state machine
- idempotent submission
- acknowledgement handling
- partial fills
- cancellations
- rejection handling
- reconciliation
- reconnect recovery
- duplicate-order protection
- execution metrics

Required order states:
CREATED → SUBMITTED → ACKNOWLEDGED → PARTIALLY_FILLED → FILLED/CANCELLED/REJECTED

TradingEngine must not contain broker-specific code.

Do not enable live trading.

PaperAdapter must behave like the future live adapter as closely as possible.
```

PHASE 09 — Backtesting + Paper Trading

```
Delegate to Codex CLI.

Build an event-driven backtester and paper-trading loop.

Requirements:
- realistic costs
- configurable slippage
- timestamp correctness
- no look-ahead bias
- no survivorship bias where the data source permits point-in-time universes
- deterministic strategy execution
- walk-forward/out-of-sample support where appropriate
- trade journal compatibility

Metrics:
- net P/L
- expectancy
- profit factor
- drawdown
- trade frequency
- average trade
- cost contribution
- slippage sensitivity
- regime performance

The same strategy interfaces must run in backtest and paper modes.

Do not optimize solely for historical return.
```

PHASE 10 — Qwen3 Integration

```
Delegate to Codex CLI.

Integrate Qwen3 as the PRIMARY MARKET ANALYST.

Qwen3 must receive structured feature summaries and candidate context, not unrestricted raw tick streams.

Define a strict JSON schema for:
- thesis
- evidence
- confidence/context fields
- invalidation conditions
- risks
- suggested action
- model/version metadata
- latency
- prompt/version identifier

Qwen3 is an evidence provider, not the final authority.

Implement:
- timeout
- retry policy
- model unavailable fallback
- malformed output handling
- logging
- token/latency metrics
- deterministic validation of model output

Never allow malformed AI output to bypass risk rules.
```

PHASE 11 — DeepSeek-R1 Challenger Layer

```
Delegate to Codex CLI.

Integrate DeepSeek-R1 as the adversarial/challenger reasoner.

DeepSeek-R1 receives the same structured candidate context plus Qwen3's analysis.

Its job:
- challenge assumptions
- identify contradictory evidence
- detect weak setups
- identify missing evidence
- highlight invalidation conditions
- stress-test the thesis

Do NOT implement simple majority voting.

The deterministic Trade Gate remains final.

Implement structured output validation, timeout/fallback, model metrics and audit logging.

Create tests where Qwen3 is bullish but DeepSeek-R1 identifies a risk, and verify that deterministic rules
decide the final action.
```

PHASE 12 — Deterministic Trade Gate + AI Council

```
Delegate to Codex CLI.

Implement the final decision pipeline:

1 deterministic candidate gate
2 risk/capital/cost gate
3 Qwen3 analysis
4 DeepSeek-R1 challenge
5 deterministic final TradePlan
6 execution authorization
7 Position Watcher ownership

The final TradePlan must contain:
- instrument
- side
- quantity
- entry context
- expected edge
- estimated costs
- risk limits
- stop/protection
- invalidation
- strategy
- AI evidence references
- decision reason
- approval mode

Ensure the AI cannot directly submit an order.

Add complete audit logging.
```

PHASE 13 — Database, Journal & Performance

```
Delegate to Codex CLI.

Finalize SQLite schema with WAL and safe transaction handling.

Core entities:
capital_state
positions
orders
trades
candidates
features
ai_analyses
trade_plans
risk_events
strategy_versions
model_calls
execution_metrics
system_events

Implement:
- trade journal
- P/L calculations
- performance aggregation
- strategy-level metrics
- model latency metrics
- execution latency metrics
- audit trail
- restart recovery

Ensure all important decisions are reproducible from stored records.
```

PHASE 14 — FastAPI Backend + API Contracts

```
Delegate to Codex CLI.

Implement the approved FastAPI backend and API contracts.

Expose safe read/query APIs for:
- dashboard summary
- market scanner
- candidates
- positions
- orders
- trades
- P/L
- AI analysis
- performance
- backtests
- risk
- system health
- logs

Separate read APIs from mutation/execution APIs.

Protect live-trading endpoints behind explicit approval controls.

Add API tests and OpenAPI documentation.

Do not expose unrestricted arbitrary order execution through a generic endpoint.
```

PHASE 15 — Dashboard Implementation

```
Act as UI integration lead and delegate implementation to Codex CLI.

Build the React + TypeScript + Tailwind + shadcn trading terminal according to UI_UX_SPEC.md.

Prioritize:
- Command Center
- Market Scanner
- Candidate Feed
- Position Monitor
- Orders
- AI Analysis
- Risk/Capital
- System Health

Use realistic mock/paper data first.

Clearly label:
SYSTEM FACT
QWEN3 ANALYSIS
DEEPSEEK-R1 CHALLENGE
FINAL DETERMINISTIC GATE

Never make AI output visually appear guaranteed.

After implementation, run browser verification and compare the result against UI_UX_SPEC.md.
```

PHASE 16 — Integration, Failure Injection & QA

```
You are the release QA commander.

Do not add new features unless required to fix a verified defect.

Run:
- unit tests
- integration tests
- API tests
- UI tests
- backtest regression tests
- paper-trading tests
- restart/recovery tests

Inject failures:
- market-data disconnect
- broker disconnect
- stale data
- model timeout
- Qwen3 unavailable
- DeepSeek-R1 unavailable
- malformed AI output
- partial fill
- rejected order
- duplicate order attempt
- database failure
- process crash
- laptop restart
- inconsistent local/broker state

Verify that hard risk protection remains deterministic and independent of LLM availability.

Produce a release-readiness report.
```

PHASE 17 — Performance & Latency Optimization

```
Act as performance engineer.

Measure separately:
- market data arrival
- normalization
- feature computation
- candidate detection
- decision preparation
- Qwen3 latency
- DeepSeek-R1 latency
- deterministic gate latency
- order submission
- broker acknowledgement
- fill latency
- Position Watcher response

Do not claim an internal latency target is equivalent to end-to-end execution speed.

Optimize only measured bottlenecks.

Protect correctness and risk controls.

Produce before/after benchmarks and document hardware assumptions.
```

PHASE 18 — Paper Trading Validation

```
Run the complete system in paper mode.

No real orders.

Validate:
- candidate quality
- entry/exit behavior
- costs
- slippage assumptions
- trade frequency
- false positives
- false exits
- model disagreement
- watcher behavior
- system uptime
- recovery behavior

Record all trades and decisions.

Do not change thresholds simply to make the paper results look better.

Produce a fixed-period validation report with reproducible metrics and identified weaknesses.
```

PHASE 19 — Controlled ■100 Live Readiness

```
This is a human approval gate.

Do NOT enable live trading automatically.

Prepare a live-readiness package:
- broker adapter configuration checklist
- API credential checklist
- paper/live environment separation
- capital configuration
- ■100 initial capital
- ■300 autonomy threshold
- emergency stop procedure
- position watcher verification
- order reconciliation verification
- audit logging verification
- backup/recovery procedure
- manual approval procedure

Require explicit human confirmation before any live order capability is enabled.

If any critical test fails, remain in paper mode.
```

PHASE 20 — ■100 Controlled Live Experiment

```
Operate only under the approved live experiment rules.

Initial capital: ■100.
Autonomous threshold: ■300.
No increase in capital without explicit human approval.

Monitor:
- net P/L after costs
- execution quality
- slippage
- rejected orders
- watcher response
- model latency
- model disagreement
- trade frequency
- failure events

Do not optimize the system during an active experiment without recording the strategy version and reason.

Produce periodic experiment reports.

Never interpret short-term profit as proof of a robust strategy.
```

PHASE 21 — ■300 Autonomous Mode

```
Only enter this phase after documented validation and human approval.

Enable autonomous operation only up to the configured ■300 threshold.

Maintain:
- deterministic risk limits
- capital caps
- circuit breakers
- independent Position Watcher
- full audit logging
- automatic recovery
- human override

Above ■300, require explicit human approval.

Do not silently raise the threshold.
```

PHASE 22 — Learning & Continuous Improvement

```
Design the learning loop without allowing uncontrolled self-modification.

Use historical trade data to analyze:
- strategy performance
- regime performance
- feature usefulness
- false positives
- false exits
- AI reasoning quality
- execution quality
- cost leakage

Any strategy/model/prompt change must create a new version.

Require backtest + out-of-sample/paper validation before production use.

The system may recommend changes; it must not silently rewrite live trading rules.
```

PHASE 23 — Final Production Hardening

```
Prepare the project for long-term personal operation.

Verify:
- startup/restart
- database integrity
- logs
- backups
- configuration
- secrets handling
- broker isolation
- live/paper separation
- health monitoring
- UI reliability
- API security
- dependency pinning
- reproducible builds
- documentation

Create:
RUNBOOK.md
OPERATIONS.md
INCIDENT_RESPONSE.md
BACKUP_RECOVERY.md
RELEASE_CHECKLIST.md

Produce final architecture and operational documentation.
```

PHASE 24 — Completion Gate

```
You are the final Project Commander.

Review the entire repository against:
- ARCHITECTURE.md
- PROJECT_SPEC.md
- TRADING_RULES.md
- UI_UX_SPEC.md
- TEST_PLAN.md
- FAILURE_HANDLING.md
- BROKER_ADAPTER_SPEC.md
- DEVELOPMENT_PLAN.md

Verify every phase has evidence of completion.

Report:
1. implemented features
2. tested features
3. known limitations
4. unresolved risks
5. paper/live status
6. current capital-control configuration
7. model configuration
8. broker status
9. operational readiness
10. recommended next engineering tasks

Do not declare the system financially successful merely because it made profits.

The project is complete only when the engineering, safety, testing, UI, operational and documentation gates are satisfied.
```

Reusable Antigravity Prompts

Use these between phases whenever needed.

REVIEW ONLY

```
Review the current repository against the approved architecture and current phase requirements. Do not edit files. Identify deviations, defects, missing tests, hidden coupling, performance risks, security risks, and undocumented behavior. Return findings grouped by Critical, High, Medium and Low. Provide exact file references and recommended fixes.
```

CODEX DELEGATION

```
Inspect the current phase requirements and create a single bounded Codex CLI implementation task. Include exact files/modules allowed to change, acceptance criteria, tests required, prohibited scope, and the command(s) Codex should run to verify its work. Do not let Codex modify architecture documents unless explicitly requested.
```

POST-CODEX REVIEW

```
Codex has completed the assigned task. Review the git diff and test results. Verify architecture compliance, correctness, edge cases, failure handling, performance, and security. Do not make unrelated improvements. If defects exist, produce a precise Codex fix task. If the phase passes, update TASKS.md and the phase checkpoint.
```

PHASE CHECKPOINT

```
Close the current phase only if all acceptance criteria are satisfied. Record implementation evidence, tests run, known limitations, decisions, and the exact next phase. Do not silently carry unfinished work forward. If the phase is not complete, keep it open and identify blockers.
```

LIVE-TRADING SAFETY

```
Treat any request involving live order submission, broker credentials, capital increases, or removal of risk limits as a human approval gate. Do not automatically enable live trading, raise capital limits, disable the Position Watcher, bypass deterministic risk checks, or allow an LLM to submit an order directly.
```

Final note

The strongest workflow is not 'Antigravity does everything'. It is **Antigravity orchestrates** → **Codex implements** → **Antigravity verifies** → **user approves**. Antigravity can use terminal orchestration and subagents, but Codex remains a separate coding worker. Keep the one-writer rule and phase gates throughout the project.